

Full-Stack Engineer Take-Home Assignment

1. Task

Build a chat web application (locally) where a signed-in user can ask questions about the revenue and income of U.S. public companies. Every answer must be grounded in structured financial data held in a SQL database: the assistant answers by querying the database, and the tool call must be visibly rendered in the UI as it happens. When the answer involves multiple companies, years, or metrics, it must render as a proper table or chart in the chat.

The defining requirement is no hallucination. When the data needed to answer a question is not available, the assistant must say so clearly rather than invent figures.

Beyond correctness, this assignment is also graded on UI/UX. The chat experience should feel clean, responsive, and polished, not just functional.

2. What you will build

Area	Requirement
Chat application	A React + TypeScript frontend and a backend API (Python or TypeScript) where a signed-in user can chat with an assistant about company financials.
Streaming	Responses stream token-by-token. The SQL tool call must be visibly rendered as it happens, and the final answer must render Markdown properly, including tables and charts where the data calls for them.
Grounding	Answer only from the SQL database. If the data needed isn't available, say so plainly, never invent numbers.
Usage limits	Each user has a configurable spending limit (default \$1) that resets on a fixed interval (default 1 hour). Exceeding it must be handled cleanly.
Authentication	Users register and log in. Each user only sees their own conversations.
Chat management	Users can stop a response mid-generation, revisit past conversations, and delete a conversation.

3. Baseline scenarios your app must handle

As a baseline, the following scenarios must all work correctly.

S1: Ask about a company and year covered by the data (e.g. "What was Apple's net income in 2023?")

Expected: correct streamed answer, with the SQL tool call visibly rendered and the final answer properly formatted (including tables or charts when the answer covers multiple data points).

S2: Ask about a company or year not covered by the data

Expected: the assistant clearly states the data isn't available. No invented numbers.

S3: Start a response and click Stop halfway

Expected: the partial message is saved correctly in the conversation history, and its cost is still deducted from the user's usage limit.

S4: Exceed the usage limit

Expected: further requests are rejected with a clear, friendly message, not a raw error. For faster testing, we recommend temporarily setting the limit to \$0.001 with a 3-minute reset window.

S5: Refresh the browser mid-conversation

Expected: chat history reloads in the correct order, with no duplicate or missing messages.

S6: Delete a conversation

Expected: the user is asked for confirmation first, and the conversation is fully removed afterward.

4. Provided Data

You receive a folder with the raw source data.

Item	Description
data/financial_data.sql	A PostgreSQL dump (schema plus COPY data). Income-statement data for 48 U.S. companies, 2022-2025 (192 rows). Columns: company, ticker, sector, year, revenue, gross_profit, operating_income, net_income (all in USD).
OpenAI API key	Provided separately. Use it for the LLM and for tool-calling.

Data coverage (read this carefully)

The data covers 48 companies for 2022-2025 only. Any question about a company, year, or metric outside this coverage has no grounding data. By design, this is what exercises the no-hallucination requirement (see scenario S2).

5. Tech stack

- **Frontend:** React + TypeScript (required). Any UI frameworks are accepted.
- **Backend:** Python or TypeScript. FastAPI, Express.js, or NestJS are preferred.
- **LLM:** OpenAI (key provided). Any LLM frameworks (e.g., LangChain, Vercel AI sdk, LlamaIndex) are accepted.
- **Databases:** PostgreSQL
- **Cache storage:** Redis

6. Data infrastructure: run it locally with Docker Compose

Bring up PostgreSQL and Redis yourself with Docker Compose.

Load the SQL

data/financial_data.sql is a standard pg_dump-style file: a DDL prelude followed by a COPY FROM stdin block of tab-separated rows. Restore it into your Postgres with psql, or a small loader that runs the DDL and then streams the COPY block.

Usage tracking

Track each user's spend within the current fixed window, and to enforce the limit and reset interval described in section 2. Both the limit and the reset interval must be configurable.

7. Authentication

Implement simple registration and login. Keep the auth and data-access layers clean; we may build on it during a live session.

8. Deliverables

1. A Git repository containing your source code, a docker-compose for the local data stack, and any seed or loader scripts.
2. A README with setup steps: how to bring up Postgres and Redis, how to load the data, how to run the app, and how to configure the usage limit and reset interval.
3. The running application correctly handles scenarios S1 to S6 above.

9. Grading Rubric

Submissions are scored against the criteria below

Criteria	Weight	What we look for
UI/UX & chat experience	25%	Clean, polished UI; smooth streaming; tables/charts render correctly.
Correctness & baseline scenario coverage (S1-S6)	40%	Answers grounded in SQL results, no invented figures (S2); all six baseline scenarios pass end-to-end, including stop/resume, usage limits, reload, and delete.
Engineering and code quality	25%	Visible tool-call streaming, clean auth/data isolation, correct usage limits, well-separated code.
Setup & documentation	10%	README lets a reviewer run Postgres/Redis, load data, and run the app with no guesswork.

10. Note

Use AI assistants, but own your code

You may use AI coding tools. But you must fully understand, and be able to explain and extend, every part of your codebase. In a live session we will ask you to walk through your design and then implement a small extension on top of it.

Budget Limit

The provided OpenAI API key is capped at US\$10 which we think is plenty for this task. You are also free to use another AI provider you already have, if you prefer; it is your choice.